Day 10 Worksheet

OOP III: Advanced OOP

Answer every question below. Write and run your code where asked - do not just reason about it on paper. None of these repeat an example already worked out on the slides - they use fresh scenarios to test the same concepts.

Question 1 *(Encapsulation)*

Write a class Thermostat with a private attribute __temperature, set through the constructor. Add a method increase_temp(amount) that raises the temperature but never lets it go above 30, and a method get_temperature() to read the current value. Show that accessing thermostat.__temperature directly from outside fails.

Question 2 *(Encapsulation)*

In 2 to 3 sentences, explain why a protected attribute (_name) is not actually enforced by Python, while a private attribute (__name) is at least partially enforced. What is the actual mechanism behind the private one?

Question 3 *(Nested Classes)*

Write a class Restaurant with a nested class MenuItem, where MenuItem takes a name and a price and has a display_price() method. Create a MenuItem directly using Restaurant.MenuItem(...), without ever creating a Restaurant object first.

Question 4 *(Association / Aggregation / Composition)*

For each of the following pairs, decide whether the relationship is association, aggregation, or composition, and explain why in one sentence: (a) a Human and their Heart, (b) a Customer and an Order they place, (c) an Orchestra and its Musicians.

Question 5 *(Composition)*

Write a Laptop class that creates and owns a Battery object inside its own constructor. The Laptop should have a check_power() method that calls a method on its Battery.

Question 6 *(Abstract Classes)*

Write an abstract class Instrument with an abstract method play(). Create two subclasses, Guitar and Piano, each implementing play() with its own message. Put one of each in a list and call play() on each in a loop.

Question 7 *(Abstract Classes)*

What error do you get if you try to create an object directly from an abstract class that still has an unimplemented abstract method? Try it with your Instrument class from question 6 (by removing play() from one subclass) and paste the exact error message.

Question 8 *(Data Classes)*

Rewrite the following class using @dataclass: a class Movie with attributes title, director, and duration_minutes. Show that print(movie) gives a readable output automatically.

Question 9 *(Keyword Arguments)*

Create an object of your Movie class from question 8 using keyword arguments, passing the three values in a different order than they were defined. Confirm it still works correctly.

Question 10 *(Putting It Together (Harder))*

Design an abstract class Employee with an abstract method calculate_pay(). Create two subclasses, SalariedEmployee and HourlyEmployee, each with their own private salary/rate attribute and their own calculate_pay() logic. Put a mix of both types in one list and print each employee's pay in a loop.

Question 11 *(Putting It Together (Harder))*

Design an abstract class Equipment with an abstract method usage_instructions(). Create two subclasses, Treadmill and Weights, each implementing usage_instructions() differently. Then build a Gym class that uses composition to hold a private list of Equipment objects, with a method to add equipment and a method to list all equipment by calling usage_instructions() on each. Also create a Member using @dataclass with name and membership_id, and create one Member using keyword arguments.